

Recommender Systems at Scale, and PCA

Related Items, Deep Content-Based Filtering, Large Catalogues, Ethics, and Principal Component Analysis

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 8 · Part 2

From a toy matrix to millions of items

Part 1 gave us

- Content-based prediction: $\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)}$ with known item features.
- Collaborative filtering: learn $\mathbf{x}$, $\mathbf{w}$, b together from the ratings alone.
- Binary labels, mean normalization, a `GradientTape` loop.

Real systems have millions of users and items, rich side information about both, and a few hundred milliseconds to answer. Today: how they cope — and a dimensionality-reduction method that grew out of the same idea of compact representations.

Today

1. Finding related items
2. Collaborative vs. content-based filtering
3. Deep learning for content-based filtering
4. Recommending from a large catalogue
5. Ethical use of recommender systems
6. Principal component analysis (optional)

Finding Related Items

“Because you ordered *Pepper Soup*” . . .

Use the learned features

Collaborative filtering gave every dish a feature vector $\mathbf{x}^{(i)}$. Dishes liked by the same people end up close together, so the items most related to dish i are those minimising

$$\left\|\mathbf{x}^{(k)} - \mathbf{x}^{(i)}\right\|^2 = \sum_{l=1}^{n} \left(x_l^{(k)} - x_l^{(i)}\right)^2.$$

Key idea

No cuisine labels, no recipes: relatedness comes entirely from co-ratings. This is “customers who liked this also liked” made precise.

Distances from *Pepper Soup*

dish	$\|\mathbf{x}^{(k)} - \mathbf{x}^{(3)}\|^2$
Chilli Noodles	0.091
Chicken Curry	0.856
Mango Ice Cream	7.573
Ginger Cake	7.617

Computed from the features learned in Part 1’s animation. *Chilli Noodles* is nearest; the sweet dishes are more than eighty times as far away.

Collaborative vs. Content-Based Filtering

Two sources of information

	collaborative filtering	content-based filtering
uses	ratings of users who rated similarly	features of the user and of the item
item features	learned from ratings	given: cuisine, price, spice level, ingredients, photos
new item	cannot be recommended until rated (*cold start*)	recommended at once from its features
new user	needs a few ratings (or mean normalization)	needs profile features (age, country, stated interests)
strength	discovers tastes no feature describes	works from day one; uses side information

Features for content-based filtering

User $\mathbf{x}_u$: age, location, past ratings per cuisine, dishes ordered.

Dish $\mathbf{x}_m$: cuisine, price, spice level, reviews, average rating.

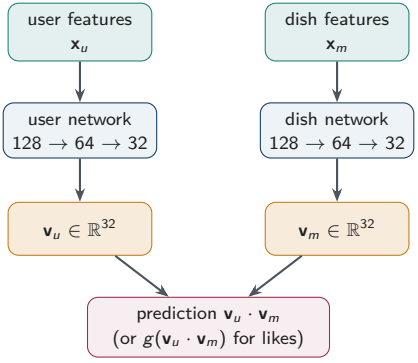
The two vectors can have different lengths.

Key idea

Large systems use both: content features for new items and new users, collaborative signals once interaction data accumulates.

Deep Learning for Content-Based Filtering

Two towers



The idea

One network maps user features to $\mathbf{v}_u$, another maps dish features to $\mathbf{v}_m$, of the *same* length. Predict with their dot product — Part 1's $\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)}$, both sides now computed.

Training

Both towers are trained together on one cost:

$$J = \sum_{r(i,j)=1} \left(\mathbf{v}_u^{(j)} \cdot \mathbf{v}_m^{(i)} - y^{(i,j)} \right)^2 + \text{penalty}$$

Related dishes: small $\|\mathbf{v}_m^{(k)} - \mathbf{v}_m^{(i)}\|^2$, precomputed overnight for the whole catalogue.

Two towers in TensorFlow

```
user_NN = Sequential([Dense(256, activation='relu'),
                      Dense(128, activation='relu'),
                      Dense(32)])
item_NN = Sequential([Dense(256, activation='relu'),
                      Dense(128, activation='relu'),
                      Dense(32)])
u_in = Input(shape=(n_user_feats,))
m_in = Input(shape=(n_item_feats,))
vu = tf.linalg.l2_normalize(user_NN(u_in), axis=1)
vm = tf.linalg.l2_normalize(item_NN(m_in), axis=1)
out = Dot(axes=1)([vu, vm])
model = Model([u_in, m_in], out)
model.compile(optimizer=Adam(), loss=MeanSquaredError())
```

What is new

Two `Sequential` networks, combined with the functional API: `Model` takes two inputs and one output. `Dot` computes $\mathbf{v}_u \cdot \mathbf{v}_m$ for each pair.

Normalizing the vectors

`l2_normalize` gives every vector length 1, so the dot product is the cosine of the angle between them — a scale-free similarity that tends to train more stably.

Recommending from a Large Catalogue

catalogue: millions of items

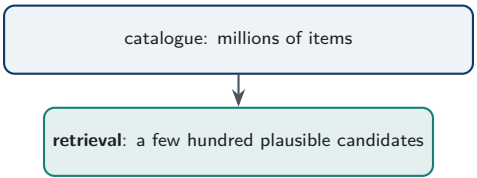
The problem

Scoring every one of millions of items with a deep network, for every user request, in a fraction of a second, is too slow and too expensive.

Key idea

Retrieval must be fast and high-recall; ranking must be accurate. Splitting the two lets each be optimised for its own job.

Retrieval and ranking



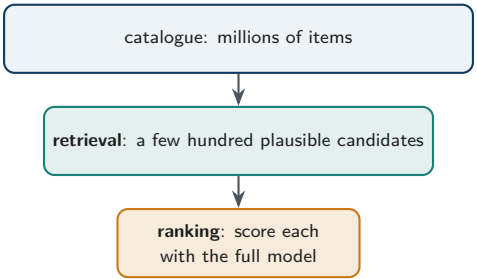
Step 1: retrieval

Cheaply generate a broad candidate list: the 10 items most similar to each of the user's last 10 ordered; the most popular dishes in their three favourite cuisines; top dishes in their area. Remove duplicates and restaurants that are closed or too far away.

Key idea

Retrieval must be fast and high-recall; ranking must be accurate. Splitting the two lets each be optimised for its own job.

Retrieval and ranking



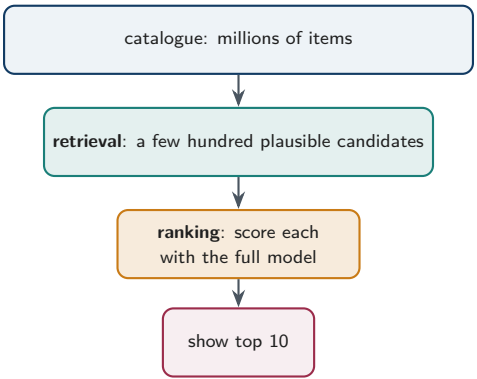
Step 2: ranking

Run the two-tower model on just these candidates. $\mathbf{v}_m$ for every item is precomputed; only $\mathbf{v}_u$ is computed per request — one network evaluation, then a few hundred dot products.

Key idea

Retrieval must be fast and high-recall; ranking must be accurate. Splitting the two lets each be optimised for its own job.

Retrieval and ranking



Step 3: display

Show the highest-scoring items. Retrieving more candidates improves the recommendations but slows them down: tune the number retrieved with offline experiments that measure the trade-off.

Key idea

Retrieval must be fast and high-recall; ranking must be accurate. Splitting the two lets each be optimised for its own job.

Ethical Use of Recommender Systems

What should a recommender optimise?

Possible goals

- items the user is most likely to rate highly;
- items most likely to be bought;
- advertisements most likely to be clicked — and paid for;
- items that generate the most profit;
- videos that maximise watch time.

When the goal diverges from the user's good

- Watch-time maximisation can amplify outrage, conspiracy and extreme content, because it engages.
- Advertising auctions can reward the businesses with the highest margins, not the best service.
- Feedback loops: what is recommended gets clicked, so gets recommended more, narrowing what users ever see.

Choosing a metric is choosing a value

"Maximise engagement" sounds neutral. It is a decision that the system should show people whatever holds their attention, whether or not they are better off for it.

Practices that help

- Be transparent with users about why an item is shown.
- Filter out problematic content and exploitative sellers.
- Measure long-term satisfaction, not only clicks.
- Inject diversity; audit who and what is never recommended.

Principal Component Analysis

Reducing the number of features

Why

Data with 50 or 1000 features cannot be plotted. Often the features are strongly correlated — a car's length and width, a country's GDP and its energy use — so the data lies close to a lower-dimensional surface.

Principal component analysis (PCA)

Find new axes — the **principal components** — such that projecting the data onto the first k of them keeps as much of its *variance* (spread) as possible. Replace each $\mathbf{x} \in \mathbb{R}^n$ by its k coordinates $z_1, \ldots, z_k$, with $k \ll n$.

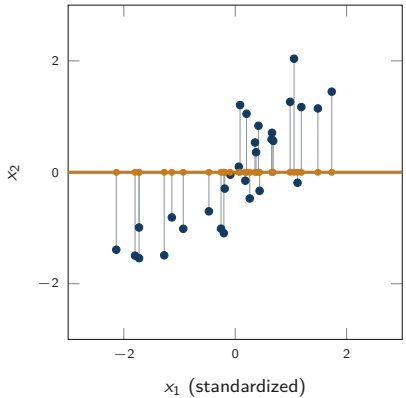
Uses

- **Visualisation**: reduce to 2 or 3 dimensions and plot.
- **Compression**: store k numbers instead of n.
- Faster training (less common now).

Key idea

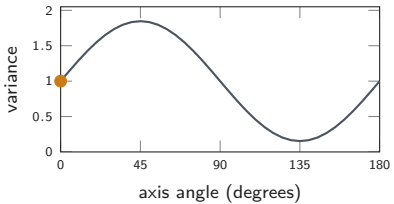
Like collaborative filtering, PCA finds a compact representation in which a few learned numbers summarise many observed ones. Unlike it, PCA is unsupervised and has an exact solution.

Searching for the best axis



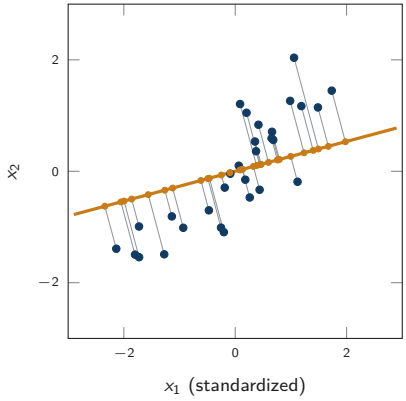
Project onto a line

candidate axis: angle 0°, variance of projections = 1.000



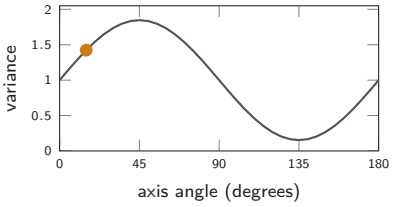
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



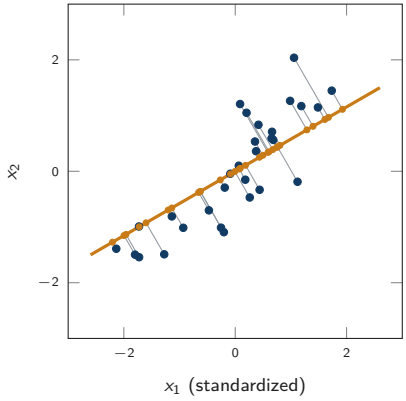
Project onto a line

candidate axis: angle 15°, variance of projections = 1.423



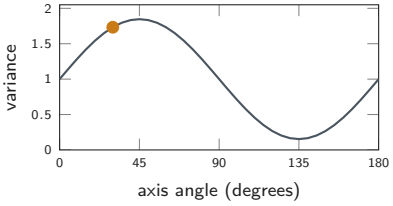
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



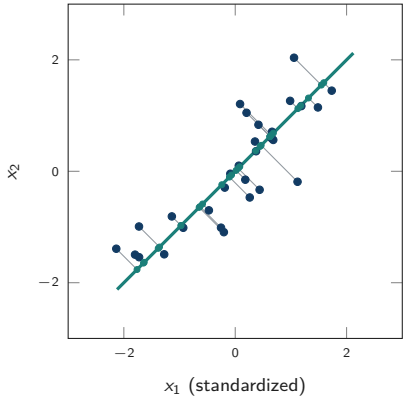
Project onto a line

candidate axis: angle 30°, variance of projections = 1.733



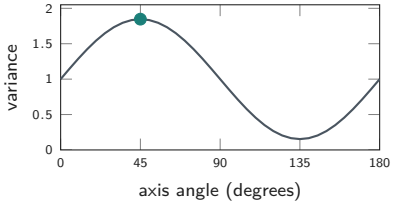
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



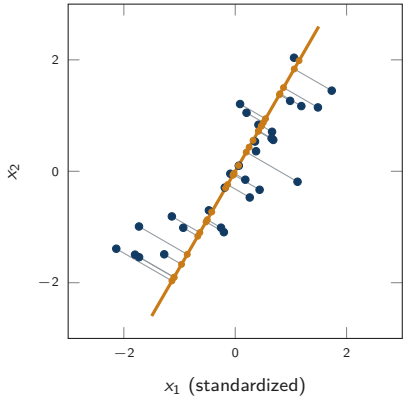
Project onto a line

first principal component: angle 45°, variance of projections = 1.847



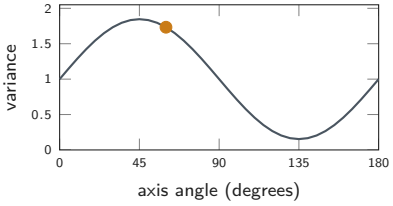
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



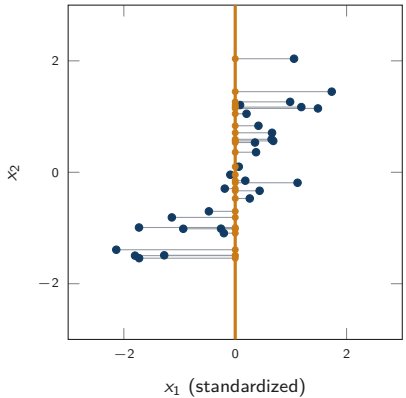
Project onto a line

candidate axis: angle 60°, variance of projections = 1.733



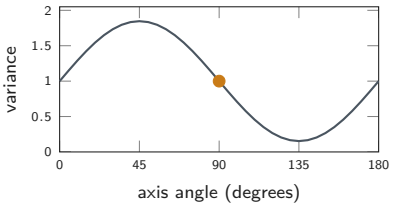
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



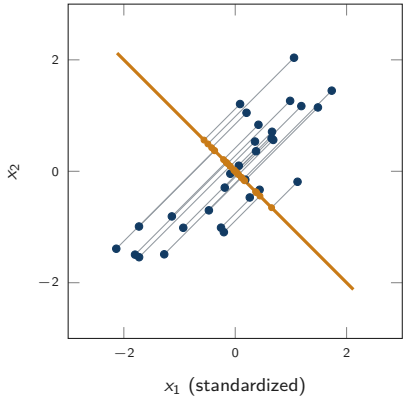
Project onto a line

candidate axis: angle 90°, variance of projections = 1.000



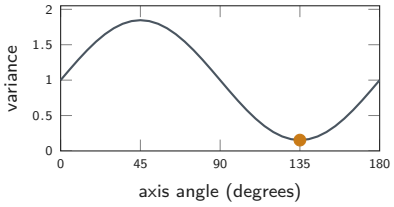
30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

Searching for the best axis



Project onto a line

candidate axis: angle 135°, variance of projections = 0.153



30 points, two correlated features, standardized. Each grey segment joins a point to its projection on the axis.

The first principal component

The variance of the projections is largest, 1.847, at 45° — the direction along which the points are most spread. It is the first principal component, $\mathbf{u}_1 = (0.7071,\ 0.7071)$. The perpendicular axis, at 135°, keeps only 0.153.

Variance explained

The total variance is 2 (two standardized features). The first component keeps $1.847/2 = 92.3\%$ of it, the second 7.7%. One number per point instead of two loses less than a tenth of the information.

Worked example: one point

$\mathbf{x} = (-1.726,\ -1.542)$. Its coordinate on the new axis:

$$z = \mathbf{x} \cdot \mathbf{u}_1 = -2.311$$

Reconstruction — the best approximation of $\mathbf{x}$ from z alone:

$$\hat{\mathbf{x}} = z\,\mathbf{u}_1 = (-1.634,\ -1.634)$$

PCA is not linear regression

Regression minimises *vertical* distances to predict x_2 from x_1. PCA minimises *perpendicular* distances — the grey segments — and treats all features alike. Average squared reconstruction error here: 0.153, exactly the variance left in the second component.

PCA, step by step

1. **Centre**: subtract each feature's mean. (**Scale** too, if the features have different units.)
2. Compute the covariance matrix $\Sigma = \frac{1}{m} \sum_i \mathbf{x}^{(i)} \mathbf{x}^{(i)\mathsf{T}}$.
3. Find its eigenvectors $\mathbf{u}_1, \mathbf{u}_2, \ldots$, ordered by eigenvalue $\lambda_1 \geq \lambda_2 \geq \cdots$.
4. Project: $z_j = \mathbf{x} \cdot \mathbf{u}_j$ for $j = 1, \ldots, k$.

λ_j is the variance captured by component j.

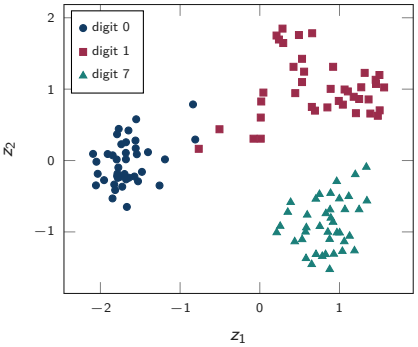
Our example

Eigenvalues of Σ: 1.847 and 0.153. They are exactly the two variances the animation found at 45° and 135°: the search over angles and the eigen-decomposition give the same answer, but the second needs no search.

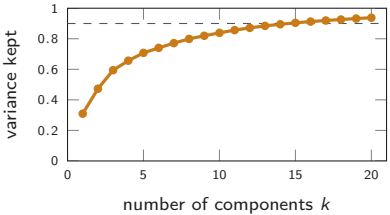
Forgetting to centre or scale

Without centring, the first "component" points at the mean rather than along the spread. Without scaling, a feature measured in millimetres dominates one measured in metres simply because its numbers are bigger.

PCA on handwritten digits: 64 features to 2



40 of each digit shown; PCA fitted on all 539 images of 0, 1 and 7



Each 8×8 image is a point in $\mathbb{R}^{64}$. The first two components keep 47.3% of the variance — yet the three digits already form three clear groups, found without labels. Keeping 90% needs 15 components; 95% needs 23.

PCA in code, and when not to use it

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

Xs  = StandardScaler().fit_transform(X)
pca = PCA(n_components=2).fit(Xs)
pca.explained_variance_ratio_   # e.g. [0.92, 0.08]
Z     = pca.transform(Xs)             # (m, 2)
X_hat = pca.inverse_transform(Z)      # (m, n)
```

PCA is not a cure for overfitting

Reducing features with PCA to avoid overfitting throws away variance without asking whether it predicts y — PCA never looks at the labels. Use regularization (Week 3) instead.

Good uses today

Visualising high-dimensional data (often as a first look before t-SNE or UMAP); compressing data for storage; decorrelating features. Fit the scaler and PCA on the training set only, then apply them to new data.

Wrap-up and Assessment

Summary

What to remember

- Related items: nearest learned feature vectors.
- Collaborative filtering suffers cold start; content-based filtering uses side features and works from day one.
- Two towers: networks map user and item features to vectors whose dot product predicts the rating.
- Large catalogues: cheap high-recall retrieval, then accurate ranking.
- The objective a recommender optimises is an ethical choice.
- PCA: projections onto eigenvectors of the covariance matrix keep the most variance; centre and scale first.

Formulas

$$\text{related}(i) = \arg\min_{k \neq i} \|\mathbf{x}^{(k)} - \mathbf{x}^{(i)}\|^2$$

$$\hat{y} = \mathbf{v}_u \cdot \mathbf{v}_m, \qquad z_j = \mathbf{x} \cdot \mathbf{u}_j$$

$$\text{variance kept} = \frac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_n}$$

Next

Week 9: reinforcement learning — learning from rewards, the Bellman equation, and deep Q-networks.

Descriptive questions

1. Explain how collaborative filtering finds items related to a given item, and use the lecture's distance table to rank the dishes related to *Pepper Soup*. **[5]**
2. Compare collaborative and content-based filtering, including how each handles a new item and a new user. **[6]**
3. Describe the two-tower architecture and its cost function. Why must the two towers output vectors of the same length? **[6]**
4. Why is it useful to normalize $\mathbf{v}_u$ and $\mathbf{v}_m$ to unit length? **[4]**
5. Explain retrieval and ranking, and the trade-off in choosing how many items to retrieve. **[5]**
6. Give two ways in which optimising engagement can harm users. **[4]**

Descriptive questions (continued)

7. State the four steps of the PCA algorithm. Why must the data be centred? When must it be scaled? **[5]**
8. For the lecture's example, explain why the variance of the projections is largest at 45° and what the value 1.847 means. **[5]**
9. Project $\mathbf{x} = (1.0,\ 0.6)$ onto $\mathbf{u}_1 = (0.7071,\ 0.7071)$ and compute its reconstruction and reconstruction error. **[6]**
10. Distinguish PCA from linear regression, with a sketch. **[5]**
11. Eigenvalues $4.0, 2.5, 0.9, 0.4, 0.2$: how many components keep at least 90% of the variance? **[4]**
12. Why is PCA a poor remedy for overfitting? **[4]**

Design questions

1. **Design a news recommender.** A national news portal wants personalised front pages. Design the user and article features, the two-tower model, the retrieval stage, and safeguards against filter bubbles and sensational content. **[12]**
2. **Design for a marketplace.** An online marketplace has 5 million products and 200 milliseconds per request. Design the retrieval and ranking pipeline, what is precomputed, and how you will choose the number of candidates. **[12]**
3. **Design an objective.** A video platform's recommender maximises watch time. Propose an alternative objective that better serves users, how you would measure it, and how you would test it safely. **[10]**
4. **Design a visualisation.** You have survey data with 40 numeric answers from 2000 respondents. Design a PCA-based visualisation: preprocessing, the choice of k, and how you would explain the axes to a non-technical audience. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§12.2** — principal components analysis, including the proportion of variance explained and the importance of scaling; **§12.3** — matrix completion for recommender systems.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§12.1** — PCA derived as maximum-variance projection and as minimum-error reconstruction, the two views of this lecture.

Lab (CSE 816, Week 8): collaborative filtering; deep learning for content-based filtering; PCA and data visualization (optional).